# Algorithmique Fondamentale et Structures de Contrôle

**Thème principal :** Structures conditionnelles, boucles et contrôle du flux d'exécution

**Synthèse exécutive :** Ce cours pose les fondations de l'algorithmique impérative enseignée au premier semestre de BUT Informatique. Il formalise le passage du problème métier à sa résolution automatisée par la machine à travers la spécification rigoureuse des données, l'évaluation des conditions booléennes et la maîtrise des structures itératives (boucles bornées et non bornées). L'accent est mis sur la logique d'exécution, la gestion des cas limites et la traçabilité des états mémoire.

## Fondements de l'Algorithmique et Typage des Variables

Un algorithme est une suite ordonnée et non ambiguë d'instructions permettant de résoudre un problème en un nombre fini d'étapes. Avant toute manipulation, la machine doit allouer de l'espace pour stocker les données. Les types primitifs fondamentaux sont les entiers (Integer), les réels/flottants (Float), les booléens (Boolean) et les chaînes de caractères (String). Une variable se caractérise par son identificateur, son adresse physique en mémoire RAM, son type qui détermine le nombre d'octets alloués et les opérations autorisées, et enfin sa valeur actuelle. Une confusion fréquente chez les débutants consiste à confondre l'opérateur d'affectation (noté `<-` ou `=`) et le test d'égalité mathématique (noté `==`). L'affectation évalue d'abord l'expression située à droite, puis écrit le résultat dans la case mémoire désignée à gauche.

- Variable : emplacement mémoire nommé doté d'un type et d'une valeur.
- Typage statique vs dynamique : distinction entre la réservation explicite en mémoire et l'inférence à l'exécution.
- Affectation : opération destructrice pour l'ancienne valeur qui lie une nouvelle valeur à une variable.

***Cas concret enseignant :*** *La métaphore de la boîte aux lettres étiquetée : une variable est une boîte aux lettres portant un nom précis sur laquelle est fixé un format strict (on ne peut pas faire entrer un colis volumineux dans une fente à lettre). Si on remet un nouveau courrier dedans, l'ancien est irrémédiablement écrasé et détruit.*

## Prise de Décision : Expressions Booléennes et Alternatives

L'exécution séquentielle linéaire ne suffit pas pour résoudre des problèmes réels ; le programme doit pouvoir dévier selon l'état des données. La structure conditionnelle 'Si (condition) Alors ... SinonSi ... Sinon ... FinSi' oriente le flux d'instructions. Les conditions reposent sur des opérateurs relationnels (<, <=, >, >=, ==, !=) combinés à l'aide d'opérateurs logiques régis par les lois de De Morgan. Une attention particulière doit être portée à l'évaluation court-circuit : dans une expression `A ET B`, si `A` est faux, l'interpréteur n'évalue même pas `B`. Cette propriété est cruciale pour éviter des erreurs fatales d'exécution, notamment lors de vérifications de pointeurs nuls ou d'accès hors limites dans un tableau.

- Prédicat : expression dont l'évaluation produit obligatoirement Vrai (True) ou Faux (False).
- Opérateurs logiques : ET (conjonction), OU (disjonction), NON (négation).
- Évaluation court-circuit : optimisation où l'évaluation s'arrête dès que le résultat global est garanti.

***Cas concret enseignant :*** *Le tourniquet de métro à Anglet/Bayonne : pour franchir la barrière, la condition est (Pass_Valide ET Solde_Positif). Si le pass est périmé, le système refuse immédiatement sans même interroger la base bancaire distante, économisant du temps de calcul et évitant un plantage.*

## Structures Itératives : Boucles Déterminées et Indéterminées

La répétition est la force motrice de l'informatique. On distingue deux grandes familles d'itérations : la boucle déterministe (Pour / For) et la boucle conditionnelle (Tant Que / While). La boucle 'Pour' initialise un compteur, vérifie la borne finale et applique un pas d'incrémentation à chaque tour. La boucle 'Tant Que' est indispensable

lorsque le nombre d'itérations dépend d'un état dynamique (lecture d'un flux, saisie utilisateur, convergence numérique). Le danger absolu d'une boucle 'Tant Que' est la boucle infinie, survenant lorsque la condition reste indéfiniment vraie. Pour garantir la terminaison d'un algorithme, l'étudiant doit toujours identifier un 'variant de boucle', c'est-à-dire une quantité entière positive qui décroît strictement à chaque itération jusqu'à atteindre zéro.

- Boucle 'Pour' : boucle bornée dont le nombre d'itérations est connu à l'avance.
- Boucle 'Tant Que' : boucle non bornée avec test de condition préalable.
- Invariant et Variant de boucle : outils théoriques assurant la correction et la terminaison.

***Cas concret enseignant :*** *Le distributeur automatique de billets : il utilise une boucle 'Tant Que' (tant que la somme demandée n'est pas atteinte, distribuer un billet adapté) plutôt qu'un 'Pour 3 billets', car selon le montant et les coupures disponibles, le nombre exact d'itérations est imprévisible à l'avance.*

## ■ Questions & Interventions des Étudiants en Classe

**Question étudiant :** Pourquoi ne pas toujours utiliser une boucle 'Tant Que' si elle peut techniquement tout faire, y compris remplacer une boucle 'Pour' ?
**Réponse du professeur :** C'est une question de lisibilité, d'expressivité et de sécurité. Une boucle 'Pour' encapsule l'initialisation, le test de borne et l'incrémentation en une seule ligne. En utilisant 'Tant Que' pour un simple parcours d'indice, vous risquez d'oublier l'incrémentation en fin de bloc et de créer une boucle infinie. En partiel, utiliser un 'Tant Que' à la place d'un 'Pour' évident montre un manque de rigueur algorithmique.

**Question étudiant :** Quelle est la différence concrète entre 'A OU B' et 'A OU_EXCLUSIF B' ?
**Réponse du professeur :** En logique et en programmation, le 'OU' standard est inclusif : si A et B sont tous les deux vrais, l'expression globale est vraie. Le 'OU EXCLUSIF' (XOR) exige qu'exactement une seule des deux conditions soit vraie, mais pas les deux simultanément. Par exemple, au restaurant : 'fromage OU dessert' dans un menu économique est souvent un XOR (l'un ou l'autre mais pas les deux).

## ■ Exercice & Démonstration d'Application

**Énoncé :** Écrire l'algorithme complet permettant de calculer la moyenne des notes strictement positives saisies par l'utilisateur. La saisie s'arrête dès que l'utilisateur saisit la valeur sentinelle -1. Si aucune note valide n'a été entrée, l'algorithme doit afficher un message d'erreur approprié sans générer de division par zéro.

**Résolution pas à pas :**
1. Initialisation des accumulateurs : déclarer 'somme' à 0.0 (réel) et 'nb_notes' à 0 (entier). 2. Première lecture préalable (lecture anticipée) pour amorcer la condition de la boucle sentinelle : lire 'note'. 3. Structure de boucle : 'Tant Que note != -1 Faire'. 4. Vérification de validité interne : Si 'note > 0' Alors somme <- somme + note ; nb_notes <- nb_notes + 1 ; Sinon afficher 'Note invalide ignorée' ; FinSi. 5. Réitération de la lecture à l'intérieur de la boucle en dernière ligne : lire 'note'. 6. FinTantQue. 7. Traitement post-boucle : Si 'nb_notes > 0' Alors moyenne <- somme / nb_notes ; Afficher 'Moyenne : ', moyenne ; Sinon Afficher 'Aucune note valide saisie' ; FinSi.

**Résultat : L'algorithme gère correctement la sentinelle d'arrêt (-1), évite le crash système de la division par zéro (si -1 est saisi dès le départ), et ignore les notes aberrantes négatives autres que la sentinelle.**

## ■ Auto-Évaluation Flash d'Examen

**Q : Que vaut l'expression booléenne : NON( (5 > 2) ET (3 == 4) ) ?**

*R : Vrai (True). L'évaluation interne donne (Vrai ET Faux) = Faux. La négation NON(Faux) renvoie Vrai.*

**Q : Quel est le risque majeur d'une mauvaise gestion de l'incrémentation dans une boucle 'Tant Que' ?**

*R : La non-terminaison du programme, c'est-à-dire l'entrée dans une boucle infinie saturant le processeur et bloquant l'application.*

**Q : Pourquoi la comparaison de deux nombres réels (flottants) avec l'opérateur '==' est-elle considérée comme une faute grave en programmation ?**

*R : En raison de l'imprécision inhérente à la représentation binaire des nombres à virgule flottante (norme IEEE 754), des erreurs d'arrondi infinitésimales font que deux valeurs théoriquement identiques ne sont presque jamais strictement égales en mémoire. Il faut toujours comparer leur différence absolue à un seuil d'erreur acceptable (epsilon).*